

• MANUAL TÉCNICO · V1.0.0

Eco-Guía.

Guía Técnica de **Instalación, Compilación de APKs y Despliegue en Dispositivos** — métodos, comandos y configuración para Smartphone Móvil, Wear OS y Android TV.

MÉTODO 01

Android Studio

Despliegue directo por cable USB o Wi-Fi para desarrollo iterativo.

MÉTODO 02

APK Debug Manual

Distribución interna a evaluadores sin acceso al código fuente.

MÉTODO 03

APK Firmado Release

Build ofuscado y firmado criptográficamente para producción.

ECOSISTEMA

Smartphone · Wear OS · Android TV
Min. Android 7.0 (API 24)

1.0.0
AGOSTO 2026

Tabla de Contenidos

10 secciones
Manual exhaustivo

01	Introducción y Alcance del Documento Propósito, audiencia objetivo y ecosistema cubierto.	03
02	Requisitos Previos del Sistema y Dispositivo Hardware, sensores, software base y modo desarrollador.	04
03	Preparación Inicial del Código Fuente Clonado del repositorio, Firebase y sincronización Gradle.	07
04	Método 1 · Instalación Directa desde Android Studio Despliegue en tiempo real a dispositivo físico o emulador.	09
05	Método 2 · Generación e Instalación Manual de APKs Debug Comandos Gradle, rutas de salida e instalación por ADB o directa.	12
06	Método 3 · Generación de APK Firmado (Release) Keystore, firma V1/V2 y build optimizado para producción.	17
07	Configuración Post-Instalación en el Dispositivo Permisos críticos y ajustes especiales Xiaomi / MIUI / HyperOS.	20
08	Solución de Problemas Comunes (FAQ) Diagnóstico rápido de errores de instalación y ejecución.	23
09	Glosario Técnico Definiciones de ADB, Gradle, APK, JKS, MIUI, Wear OS y más.	25
10	Cierre · Resumen y Próximos Pasos Recapitulación de los tres métodos y referencias para profundizar.	27

— SECCIÓN 01

Introducción y Alcance del Documento

Propósito, audiencia objetivo y cobertura del ecosistema Android Eco-Guía.

Este manual técnico describe de forma exhaustiva los pasos necesarios para preparar el entorno de ejecución, generar los paquetes de instalación ejecutables (**archivos** `.apk`) e instalarlos en los diferentes dispositivos físicos y emulados que componen el ecosistema **Eco-Guía**. La guía está diseñada como referencia operativa para el equipo de desarrollo, evaluadores académicos y personal de despliegue, y cubre desde la clonación del repositorio hasta la configuración post-instalación en dispositivos Xiaomi con capas de personalización agresivas como MIUI o HyperOS.

El documento adopta una estructura **paso a paso numerada** con tres rutas de instalación complementarias, de modo que el lector pueda escoger el método óptimo según su contexto: desarrollo iterativo, distribución interna o release de producción. Cada sección incluye bloques de comandos terminal listados para copiar y pegar, tablas de referencia de rutas de salida y advertencias críticas sobre permisos del sistema operativo, restricciones de Google Play Protect y particularidades de fabricantes específicos.

Audiencia Objetivo

El público principal de este manual comprende **ingenieros Android** responsables del build y despliegue, **evaluadores QA** que reciben APKs para pruebas funcionales, **docentes y estudiantes** que participan en la revisión académica del proyecto, y **usuarios piloto** con conocimientos técnicos básicos que necesitan instalar la aplicación en sus dispositivos personales. Para los lectores sin experiencia previa con ADB o Gradle, se incluyen cajas de explicación contextual y un glosario técnico al final del documento.

Cobertura del Ecosistema

Eco-Guía es un ecosistema multi-dispositivo compuesto por tres módulos Android independientes que comparten un mismo backend de Firebase y una capa de datos geoespaciales común. Los módulos cubiertos por esta guía son:

- **Módulo Móvil** (`mobile`): aplicación principal para smartphones Android, incluye Realidad Aumentada, geofencing en segundo plano y visor de Geo-Drops.
- **Módulo TV** (`tv`): aplicación para Android TV con interfaz adaptada a control remoto y visualización panorámica de contenido.
- **Módulo Wear** (`wear`): aplicación para Wear OS con notificaciones de proximidad y accesos rápidos desde la muñeca.



Convenciones tipográficas utilizadas

A lo largo del documento se utilizan las siguientes convenciones para identificar visualmente cada tipo de contenido:

- **Bloques oscuros con barra macOS:** comandos de terminal listos para copiar.
- **Cajas verdes con check:** pasos exitosos o recomendaciones de buenas prácticas.
- **Cajas naranjas con signo de exclamación:** advertencias sobre permisos, Play Protect o configuraciones críticas.
- **Cajas rojas con X:** errores comunes y cómo resolverlos.
- **Badges circulares numerados [1] [2] [3]:** pasos secuenciales dentro de cada método.

— SECCIÓN 02

Requisitos Previos del Sistema y Dispositivo

Especificaciones mínimas de hardware, software y configuración del dispositivo antes de iniciar cualquier despliegue.

2.1 • Requisitos del Teléfono Móvil y Dispositivos Físicos

Antes de iniciar el proceso de compilación e instalación, es indispensable verificar que el hardware cumple con los requisitos mínimos del ecosistema Eco-Guía. La aplicación móvil utiliza capacidades avanzadas como Realidad Aumentada (ARCore), geolocalización de alta precisión y servicios en segundo plano, lo que impone restricciones sobre la versión de Android y los sensores disponibles. Un dispositivo que no cumpla estos requisitos puede compilar e instalar el APK, pero experimentará fallas funcionales en tiempo de ejecución que no se manifiestan como errores de compilación.

Matriz de Requisitos de Hardware y Software		
Categoría	Mínimo Requerido	Recomendado
Sistema Operativo	Android 7.0 (API 24)	Android 11 (API 30) o superior
Sensores	Cámara trasera, GPS básico	Cámara con ARCore, GPS de alta precisión + GLONASS
Conectividad	Wi-Fi para descarga inicial	Wi-Fi + Datos móviles para geofencing continuo
Memoria RAM	3 GB	6 GB o superior (para AR + mapas)
Almacenamiento	150 MB libres	500 MB libres (caché de mapas offline)
Cable USB	Cable de datos sincronizado	Cable USB 3.0 + conexión directa (sin hubs)
Computadora	8 GB RAM, Android Studio instalado	16 GB RAM, SSD, JDK 17, Gradle 8.x

2.2 • Preparación del Dispositivo Android (Modo Desarrollador)

Antes de poder instalar aplicaciones directamente o mediante comandos ADB, se debe habilitar la **depuración USB** en el teléfono. Este modo oculto de Android expone interfaces de diagnóstico que permiten a la computadora enviar comandos al dispositivo, instalar APKs sin interacción manual y leer logs del sistema en tiempo real. La activación es reversible y no afecta el funcionamiento normal

del dispositivo, pero debe desactivarse cuando no se esté desarrollando para evitar riesgos de seguridad en caso de conexión a computadoras no confiables.

1 Abrir la aplicación de Ajustes

Localiza y abre la aplicación de **Ajustes / Configuración** en el celular. En algunos dispositivos Xiaomi o Samsung esto puede llamarse "Configuración" o "Settings".

2 Navegar a "Acerca del teléfono"

Dirígete a **Acerca del teléfono** o, en algunas capas de personalización, *Información del software*. Esta sección contiene los metadatos del dispositivo: modelo, versión de Android, número de compilación y banda base.

3 Pulsar 7 veces "Número de compilación"

Localiza la opción **Número de compilación** (en Xiaomi: *Versión de MIUI / HyperOS*) y **presiónala 7 veces consecutivas** hasta ver el mensaje emergente "¡Ya eres desarrollador!". Un contador regresivo aparecerá a partir del tercer toque indicando cuántos toques faltan.

4 Regresar a Ajustes y abrir Opciones de desarrollador

Regresa al menú principal de Ajustes y navega a **Sistema** (o *Ajustes adicionales* en Xiaomi) → **Opciones de desarrollador**. Esta sección ya no estará oculta.

5 Activar las casillas de depuración

Dentro de Opciones de desarrollador, activa las siguientes casillas:

- ✓ **Depuración USB** (USB Debugging) — obligatorio.
- ✓ **Instalar vía USB** (Install via USB) — presente en Xiaomi/MIUI, obligatorio para instalación por ADB.
- ✓ **Depuración inalámbrica** (Wireless Debugging) — opcional, permite prescindir del cable una vez configurado.

! Alerta de seguridad – Depuración USB activa

Mientras la Depuración USB esté activa, cualquier computadora a la que conectes el teléfono por cable podrá ejecutar comandos ADB, leer logs, instalar aplicaciones y acceder a datos del sistema. **Desactiva esta opción cuando no estés desarrollando**, especialmente si conectas tu teléfono a computadoras públicas o de terceros. En dispositivos robados o comprometidos, la depuración USB es el vector de extracción de datos más común.

— SECCIÓN 03

Preparación Inicial del Código Fuente

Clonado del repositorio, configuración de credenciales Firebase y sincronización de dependencias Gradle.

Antes de compilar los ejecutables APK, se debe verificar que el proyecto cuente con las credenciales mínimas necesarias y que el árbol de código fuente esté correctamente sincronizado con la rama principal del repositorio. Esta sección cubre los tres pasos obligatorios: clonado del repositorio desde GitHub, verificación del archivo de configuración de Firebase en las rutas correctas y sincronización de dependencias a través de Gradle. Saltarse cualquiera de estos pasos producirá errores de compilación que pueden ser difíciles de diagnosticar si no se identifican tempranamente.

3.1 · Clonar el repositorio y situarse en la rama correcta

El código fuente de Eco-Guía se aloja en GitHub bajo la organización *Desarrollo-Web-Profesional-zrodriguez*. Para obtener una copia local limpia, ejecuta los siguientes comandos en una terminal ubicada en el directorio donde deseas que viva el proyecto. Es recomendable clonar siempre desde la rama `main` para asegurar que tienes la versión estable más reciente; las características en desarrollo se encuentran en ramas `feature/*` que no deben usarse para compilaciones de producción.



```
bash · clonar repositorio Eco-Guía

$ git clone https://github.com/Desarrollo-Web-Profesional-zrodriguez/Eco-Guia.git
$ cd Eco-Guia
$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
```

3.2 · Verificación de credenciales Firebase

El proyecto utiliza Firebase como backend de autenticación, base de datos en tiempo real y analítica. Cada módulo Android requiere su propio archivo `google-services.json` que contiene las claves API y los identificadores del proyecto Firebase. **Sin estos archivos, la compilación Gradle fallará** con el error `google-services.json missing` (ver FAQ §8.2). Asegúrate de que el archivo esté colocado en las siguientes rutas exactas dentro de tu copia local del repositorio:

Rutas Obligatorias del Archivo google-services.json

Módulo	Ruta del archivo	Estado
Móvil	Eco-Guía/mobile/google-services.json	Obligatorio
Android TV	Eco-Guía/tv/google-services.json	Obligatorio
Wear OS	Eco-Guía/wear/google-services.json	Opcional*

* El módulo Wear OS puede funcionar sin su propio archivo Firebase si se sincroniza vía la app móvil companion, pero se recomienda incluirlo para habilitar telemetría independiente.

3.3 · Sincronización de dependencias Gradle

Una vez clonado el repositorio y colocados los archivos Firebase, abre el proyecto en Android Studio (**File → Open → selecciona la carpeta Eco-Guía**) y espera a que Gradle finalice la descarga de todas las librerías declaradas en los archivos `build.gradle` de cada módulo. La primera sincronización puede tardar entre 5 y 20 minutos dependiendo del ancho de banda y la caché local de Maven. El indicador *"Gradle Sync Finished"* en la barra inferior de Android Studio confirmará que todas las dependencias se resolvieron correctamente.

i Diagnóstico – Sync fallida por dependencias

Si Gradle no logra sincronizar, las causas más comunes son: (1) falta de conexión a Internet, (2) versiones obsoletas de Android Studio que no soportan el `com.android.tools.build:gradle` requerido, o (3) conflictos de versión entre librerías. Ejecuta `./gradlew --refresh-dependencies` desde la terminal para forzar una re-descarga limpia, y verifica en el archivo `gradle/wrapper/gradle-wrapper.properties` que la versión de Gradle sea 8.x o superior.

— SECCIÓN 04 • MÉTODO 1 DE 3

Instalación Directa desde Android Studio

Despliegue en tiempo real a dispositivo físico o emulador — recomendado para desarrollo iterativo.

MÉTODO

01

DESARROLLO • TIEMPO REAL

Run & Reload desde el IDE

Este es el método más rápido para probar la aplicación en tiempo real en un teléfono físico o emulador. Compila el módulo seleccionado, transfiere el APK al dispositivo conectado y abre la aplicación automáticamente. Ideal para ciclos de desarrollo rápidos donde necesitas ver los cambios de código reflejados en segundos.

Paso 1 • Conectar el Dispositivo por USB

La conexión por cable sigue siendo el método más estable para depuración Android, especialmente cuando se trabaja con servicios en segundo plano como los que utiliza Eco-Guía (geofencing, ARCore). Aunque la depuración inalámbrica está disponible desde Android 11, el cable elimina problemas de latencia y de pérdida de paquetes que pueden causar timeouts en instalaciones grandes.

1

Conectar físicamente

Conecta el celular a la computadora mediante el cable USB. Evita usar hubs USB o extensiones; conecta directamente a un puerto de la computadora.

2

Aceptar el diálogo de depuración

En la pantalla del celular aparecerá un aviso: "*¿Permitir depuración USB de esta computadora?*" con la huella RSA de la computadora. Verifica que coincida con tu máquina.

3

Marcar "Permitir siempre" y aceptar

Marca la casilla "**Permitir siempre desde esta computadora**" y presiona **Aceptar**. Esto evita que el diálogo vuelva a aparecer en cada conexión futura con la misma computadora.

Paso 2 • Ejecutar la Aplicación desde Android Studio

Una vez aceptada la conexión, el dispositivo aparecerá automáticamente en el menú desplegable de dispositivos de Android Studio. Si no aparece, verifica que la depuración USB siga activa

(algunos dispositivos Xiaomi la desactivan tras reinicios) y ejecuta `adb devices` en la terminal para confirmar que el sistema operativo reconoce el teléfono.

1

Seleccionar el módulo a ejecutar

En la barra superior de Android Studio, localiza el menú desplegable de módulos y selecciona `mobile` (o `wear` / `tv` según el dispositivo al que vas a desplegar).

2

Seleccionar el dispositivo destino

En el menú desplegable contiguo, selecciona tu teléfono físico conectado. Los emuladores activos también aparecerán en esta lista, identificados por su nombre AVD.

3

Presionar Run (Play) o Shift + F10

Presiona el botón verde de **Run (Play)** ► en la barra superior, o utiliza la combinación de teclas `Shift + F10`. Gradle compilará el proyecto, enviará el APK al celular y lo abrirá automáticamente.



Build exitoso

Cuando Gradle finaliza la compilación exitosamente, verás en la barra inferior de Android Studio el mensaje **"BUILD SUCCESSFUL in Xs"** y la aplicación se abrirá en el dispositivo con la última versión del código. Los cambios posteriores al código pueden re-desplegarse con **Apply Changes** (Ctrl+F10) sin necesidad de reinstalar el APK completo.

— DIAGRAMA 1 · VISUAL

Flujo de Opciones de Instalación

Árbol de decisión para elegir el método de instalación adecuado según el caso de uso.

Antes de profundizar en los métodos 2 y 3, el siguiente diagrama de árbol muestra las tres rutas disponibles y sus casos de uso óptimos. La elección del método depende principalmente de dos factores: **(a) quién recibirá el APK** (desarrollador, evaluador o usuario final) y **(b) el nivel de optimización y firma criptográfica requerido**. Cada ruta produce un artefacto diferente y sigue un flujo de trabajo distinto, pero todas comparten el mismo punto de partida: el repositorio Eco-Guía clonado localmente con sus dependencias sincronizadas.



i Cómo usar este diagrama

Identifica tu caso de uso en la columna derecha (Desarrollador / QA / Usuario final) y sigue la rama correspondiente. **No es obligatorio seguir los métodos en orden** — un evaluador que recibe un APK Debug no necesita pasar por el Método 1, y un despliegue de producción puede saltarse los métodos 1 y 2 si el build firmado se ejecuta desde una máquina CI/CD. Sin embargo, los tres métodos comparten la misma base de código y los mismos prerequisites descritos en las secciones 2 y 3.

— SECCIÓN 05 · MÉTODO 2 DE 3

Generación e Instalación Manual de APKs Debug

Compilación por línea de comandos o por interfaz IDE, rutas de salida e instalación por ADB o transferencia directa.

MÉTODO

02

DISTRIBUCIÓN INTERNA · QA

APK Debug para compartir

Si deseas compartir el archivo instalador (`.apk`) con compañeros de equipo, evaluadores o maestros sin que ellos tengan que abrir Android Studio, este método genera un archivo APK Debug firmado con la clave de depuración de Android que puede transferirse por cualquier medio (Drive, WhatsApp, Telegram, correo) e instalarse en cualquier dispositivo compatible.

5.1 · Generación de los APKs

Existen dos formas de generar APKs Debug: mediante línea de comandos (recomendada para scripts de CI/CD y builds reproducibles) o mediante la interfaz gráfica de Android Studio (más conveniente para desarrolladores que ya tienen el IDE abierto). Ambas rutas producen el mismo artefacto final en la misma ruta de salida; la diferencia principal es la reproducibilidad y la posibilidad de automatizar.

Opción A · Mediante Línea de Comandos (Terminal)

Abre una terminal en la carpeta raíz del proyecto y ejecuta el comando `gradlew` con el módulo objetivo. La sintaxis varía ligeramente entre Windows (PowerShell/CMD usa `.`) y Linux/macOS (usa `./`). En ambos casos, Gradle descargará las dependencias faltantes, compilará el código Kotlin/Java, procesará los recursos Android y empaquetará el APK en la ruta de salida estándar.

```
bash · módulo móvil (smartphone)

# Windows (PowerShell / CMD)
> .\gradlew:mobile:assembleDebug

# Linux / macOS
$ ./gradlew:mobile:assembleDebug

> Task :mobile:preBuild UP-TO-DATE
> Task :mobile:compileDebugKotlin
> Task :mobile:processDebugManifest
> Task :mobile:packageDebug
BUILD SUCCESSFUL in 47s
4 actionable tasks: 4 executed
```

```
bash · módulo Android TV

# Compila el APK Debug para Android TV
$ ./gradlew:tv:assembleDebug

# Salida: tv/build/outputs/apk/debug/tv-debug.apk
```

```
bash · módulo Wear OS (smartwatch)

# Compila el APK Debug para Wear OS
$ ./gradlew:wear:assembleDebug

# Salida: wear/build/outputs/apk/debug/wear-debug.apk
```

Opción B · Mediante la Interfaz de Android Studio

Si prefieres no usar la terminal, Android Studio ofrece una ruta gráfica equivalente. Esta opción es útil cuando necesitas inspeccionar visualmente variantes de build o cuando trabajas en un equipo donde los desarrolladores no están familiarizados con la sintaxis de Gradle.

1

Navegar al menú Build

En la barra de menú superior de Android Studio, ve a: **Build → Build Bundle(s) / APK(s) → Build APK(s)**.

2

Esperar la notificación de finalización

En la esquina inferior derecha aparecerá una notificación flotante cuando termine: *"Build APK(s): APK(s) generated successfully"*. El tiempo varía entre 30 segundos y 5 minutos según la potencia de la computadora.

3

Abrir la carpeta de salida

Haz clic en el enlace azul `Locate` dentro de la notificación para abrir automáticamente la carpeta del sistema operativo que contiene el archivo APK generado.

5.2 · Ubicación de los Archivos APK Generados

Los ejecutables generados se guardan automáticamente en las siguientes rutas dentro de tu proyecto. Estas rutas son deterministas y siguen la convención estándar de Gradle para Android:

`<módulo>/build/outputs/apk/<variant>/<módulo>-<variant>.apk`. Cualquier script de CI/CD puede asumir estas rutas sin configuración adicional.

Rutas de Salida de APKs Debug Generados

Módulo	Ruta del Archivo APK Generado	Tamaño aprox.
Móvil (Smartphone)	<code>Eco-Guia/mobile/build/outputs/apk/debug/mobile-debug.apk</code>	~28 MB
Smart TV	<code>Eco-Guia/tv/build/outputs/apk/debug/tv-debug.apk</code>	~22 MB
Smartwatch (Wear)	<code>Eco-Guia/wear/build/outputs/apk/debug/wear-debug.apk</code>	~14 MB

i

¿Dónde está mi APK si la ruta no existe?

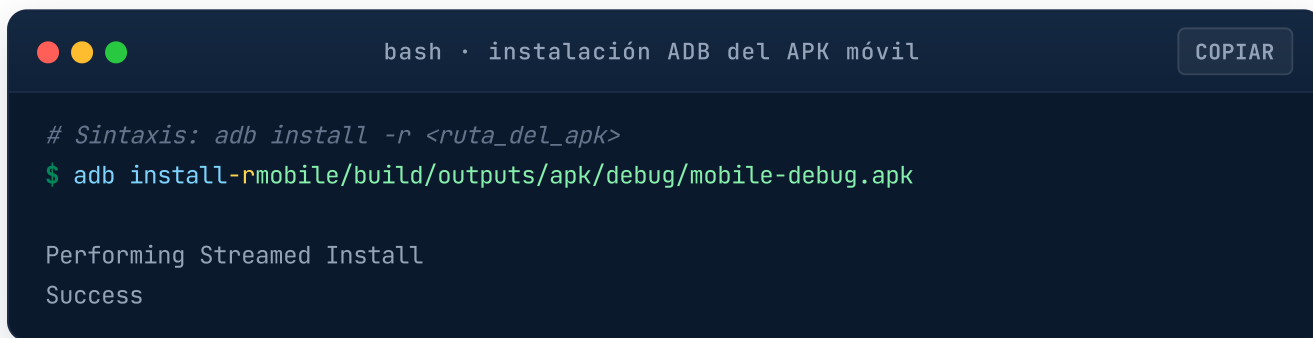
Si la carpeta `build/outputs/apk/debug/` no existe o está vacía después de ejecutar el comando, lo más probable es que la compilación haya fallado silenciosamente. Ejecuta `./gradlew :mobile:assembleDebug --stacktrace` para ver el error completo, o revisa la ventana **Build** en la parte inferior de Android Studio (no la ventana Run, que solo muestra output del runtime).

5.3 · Métodos para Instalar el APK en el Teléfono

Una vez generado el archivo `.apk`, existen dos rutas principales para instalarlo en un dispositivo Android. La elección depende de si la computadora que generó el APK tiene acceso físico al teléfono (instalación por ADB) o si el APK debe transferirse al dispositivo para instalación autónoma (instalación directa). Ambas rutas requieren que el dispositivo tenga activa la opción **"Instalar aplicaciones desconocidas"** o que confíe en el origen desde donde se descarga el archivo.

Alternativa 1 · Instalación Rápida mediante ADB (Por cable)

Si tienes el teléfono conectado a la computadora con depuración USB activa, este es el método más directo. El comando `adb install` envía el APK al dispositivo y lo instala en un solo paso. La bandera `-r` (reinstall) permite reinstalar la app conservando los datos existentes, lo cual es útil cuando estás iterando versiones durante el desarrollo.



```
bash · instalación ADB del APK móvil

# Sintaxis: adb install -r <ruta_del_apk>
$ adb install-r mobile/build/outputs/apk/debug/mobile-debug.apk

Performing Streamed Install
Success
```



Señal de éxito

Si la instalación es exitosa, la terminal responderá con la palabra `Success` en verde. Cualquier otro mensaje (`Failure [INSTALL_FAILED_...]`) indica un problema que se detalla en la sección 8 (FAQ) de este manual.

Alternativa 2 · Instalación Directa en el Teléfono (Sin computadora)

Cuando el evaluador o usuario final no tiene acceso a una computadora con ADB, el APK puede transferirse al teléfono por cualquier medio digital (cable USB, mensajería, nube) e instalarse directamente desde el administrador de archivos del dispositivo. Este método requiere aceptar diálogos adicionales de seguridad de Android porque el APK no proviene de Google Play Store.

1

Transferir el archivo APK al celular

Copia el archivo `mobile-debug.apk` mediante cable USB, o envíatelo por Google Drive, WhatsApp, Telegram o correo electrónico. En mensajería, descarga el archivo a la carpeta **Descargas**.

2

Localizar y abrir el APK

En el celular, abre la carpeta de **Descargas** o tu aplicación de **Administrador de Archivos** y toca el archivo `mobile-debug.apk`. Android mostrará un diálogo de confirmación.

3

Habilitar "Confiar en esta fuente"

Si el teléfono muestra la advertencia *"Por motivos de seguridad, tu teléfono no tiene permiso para instalar aplicaciones desconocidas de esta fuente"*:

- Toca en **Ajustes / Configuración**.
- Activa el interruptor **"Confiar en esta fuente"** (o *Permitir desde este origen*).
- Regresa y presiona **Instalar**.

4

Sobrepasar Google Play Protect

Si Google Play Protect muestra una ventana diciendo *"Aplicación bloqueada por Play Protect"* (común en APKs en desarrollo que no vienen de la tienda oficial):

- Presiona **"Más detalles"**.
- Selecciona **"Instalar de todas formas"**.

!

¿Por qué Play Protect bloquea el APK?

Google Play Protect utiliza heurísticas en la nube para identificar APKs potencialmente dañinos. Los APKs Debug firmados con la clave de depuración estándar de Android (que comparten todos los desarrolladores) disparan estas heurísticas porque **no tienen una firma criptográfica única**. Esto es completamente normal para builds de desarrollo y no representa un riesgo real. El bloqueo desaparece automáticamente cuando se usa un APK Release firmado con un keystore propio (Método 3).

— SECCIÓN 06 · MÉTODO 3 DE 3

Generación de APK Firmado (Release / Producción)

Keystore propio, firma V1 + V2 y build optimizado con ofuscación para distribución formal.

MÉTODO

03

PRODUCCIÓN · RELEASE

APK firmado con keystore propio

Para generar una versión optimizada, ofuscada y firmada con llave criptográfica propia lista para distribución final en Google Play Store o canales privados. Este método produce un APK más pequeño, más seguro y con identidad criptográfica única que permite actualizaciones posteriores firmadas por el mismo desarrollador.

El proceso de firma de Android es un mecanismo criptográfico que vincula un APK con la identidad de su desarrollador mediante un archivo `.jks` (Java Key Store) que contiene una clave privada. Esta clave **debe conservarse en secreto y respaldarse de forma segura**: si se pierde, no será posible publicar actualizaciones de la aplicación con el mismo nombre de paquete, y los usuarios existentes no podrán actualizar. La clave también se utiliza para verificar la integridad del APK en cada actualización, evitando que un atacante pueda distribuir versiones modificadas bajo el mismo nombre de paquete.

1

Abrir el diálogo "Generate Signed Bundle / APK"

En Android Studio, ve a **Build** → **Generate Signed Bundle / APK...** en la barra de menú superior. Se abrirá un asistente de varios pasos.

2

Seleccionar APK como formato de salida

Selecciona **APK** (no Android App Bundle) y haz clic en **Next**. App Bundle (.aab) es el formato recomendado para Play Store, pero APK firmado es más portable para distribución directa.

3

Configurar el Key Store (almacén de claves)

En la pantalla de configuración de la llave:

- **Si ya tienes una llave:** presiona *Choose existing...*, navega al archivo `.jks` e introduce su contraseña.
- **Si no tienes una llave:** presiona *Create new...*, elige una ruta para guardar el archivo `.jks`, define contraseñas seguras para el keystore y para la key alias, y completa los datos de la organización (nombre, organización, unidad, ciudad, país).

4

Seleccionar el tipo de compilación "release"

Selecciona el tipo de compilación `release` (no `debug`). Esto activa la ofuscación ProGuard/R8 y elimina código muerto para producir un APK más pequeño y difícil de reverse-engineer.

5

Marcar firmas V1 y V2

Marca las casillas de firma **V1 (Jar Signature)** y **V2 (Full APK Signature)**. V1 es necesaria para Android 7.0 y anteriores; V2 para Android 7.0+ y ofrece verificación más rápida. Marcar ambas maximiza la compatibilidad.

6

Finalizar y ubicar el APK

Haz clic en **Finish**. El APK optimizado se generará en `mobile/build/outputs/apk/release/mobile-release.apk`. Android Studio mostrará una notificación con un enlace `locate` para abrir la carpeta.



bash · verificación de firma del APK Release

COPIAR

```
# Verifica que el APK esté firmado correctamente
```

```
$ $ANDROID_HOME/build-tools/34.0.0/apksignerverify--verbosemobile-release.apk
```

```
Verifies
```

```
Verified using v1 scheme (JAR signing): false
```

```
Verified using v2 scheme (APK Signature Scheme v2): true
```

```
Verified using v3 scheme (APK Signature Scheme v3): false
```

```
Number of signatures verified: 1
```



Respalda tu keystore .jks

El archivo `.jks` y sus contraseñas son **la identidad criptográfica de tu aplicación**. Si pierdes el keystore, no podrás publicar actualizaciones de Eco-Guía con el mismo package name (`com.ecoguia.app`); los usuarios existentes no podrán actualizar y tendrás que publicar la app con un nombre nuevo. **Respalda el archivo en al menos dos ubicaciones independientes** (disco externo cifrado + almacenamiento en nube privado) y documenta las contraseñas en un gestor seguro como 1Password o Bitwarden.

— SECCIÓN 07

Configuración Post-Instalación en el Dispositivo

Permisos críticos, ajustes especiales Xiaomi / MIUI / HyperOS y optimización para servicios en segundo plano.

Para garantizar que todas las funciones de Eco-Guía (Realidad Aumentada, Alertas de Proximidad en Segundo Plano y Geo-Drops) funcionen al 100%, es necesario realizar una configuración inicial posterior a la instalación. Esta configuración se divide en dos partes: **(1) concesión de permisos críticos** que Android no concede automáticamente y que requieren interacción explícita del usuario, y **(2) ajustes especiales del fabricante** en dispositivos Xiaomi/Redmi/POCO donde las capas de personalización MIUI/HyperOS imponen restricciones agresivas de batería que pueden suspender los servicios en segundo plano y romper las notificaciones de geofencing.

7.1 · Concesión de Permisos Iniciales

Al abrir **Eco-Guía** por primera vez o al navegar a la pantalla de **Permisos** dentro de la app, se solicitarán tres permisos críticos que deben concederse explícitamente. Cada permiso habilita una funcionalidad central de la aplicación y, si se deniega, la funcionalidad correspondiente no fallará inmediatamente sino que se degradará silenciosamente (por ejemplo, sin cámara el visor AR mostrará una pantalla negra, sin ubicación los geofences nunca se activarán).

1

Permiso de Ubicación

Concede el permiso seleccionando **"Mientras la app está en uso"** o **"Permitir siempre"** (este último es requerido para geofencing en segundo plano, que permite que la app reciba alertas de proximidad histórica incluso cuando está cerrada).

2

Permiso de Cámara

Concede el permiso para habilitar el visor de Realidad Aumentada en los Geo-Drops. Sin este permiso, la pantalla AR mostrará un placeholder en lugar del feed de cámara en vivo.

3

Permiso de Notificaciones

Concede el permiso para que el sistema emita alertas cuando te encuentres a menos de 50 metros de un sitio histórico. En Android 13+ este permiso es explícito; en versiones anteriores se concede por defecto al instalar.

7.2 · Diagrama 2 · Flujo de Habilidadación en el Celular

El siguiente mockup visual resume el flujo completo de habilitación del dispositivo Android para recibir e instalar APKs Eco-Guía, desde la activación del modo desarrollador hasta la concesión final de permisos. Cada tarjeta representa una etapa del proceso y muestra una representación esquemática de lo que verás en la pantalla del celular.



7.3 · Diagrama 3 · Ajustes Especiales para Xiaomi / Redmi / POCO (MIUI / HyperOS)

Los dispositivos Xiaomi cuentan con optimizaciones de batería agresivas que pueden suspender los servicios en segundo plano de Eco-Guía, especialmente el servicio de proximidad geográfica que necesita ejecutarse continuamente para detectar cuándo el usuario se acerca a un sitio histórico. Sin los tres ajustes siguientes, las notificaciones de geofencing **frecuentemente fallan en dispositivos Xiaomi**, incluso cuando la app funciona perfectamente en dispositivos Samsung, Pixel o Motorola con la misma versión de Android.



CALLOUT · OPTIMIZACIÓN XIAOMI / MIUI / HYPEROS

3 ajustes indispensables para servicios en segundo plano

Sin estos tres ajustes, el sistema operativo cerrará el servicio de proximidad tras unas horas de inactividad de la pantalla, interrumpiendo las alertas geográficas. La ruta de acceso es: **Ajustes → Aplicaciones → Administrar aplicaciones → EcoGuía**.

01 / AUTOSTART

Inicio Automático

Activa esta casilla para permitir que el servicio de proximidad funcione con la app cerrada. Sin Autostart, MIUI mata el proceso al cerrar la app desde la lista de recientes.

02 / BATTERY

Sin Restricciones de Batería

Cambia la opción de Ahorro de Batería a "**Sin restricciones**" (No restrictions). La opción por defecto "Restringida" limita el uso de CPU y red en segundo plano.

03 / RESTRICTED

Permitir Ajustes Restringidos

Si la app fue instalada manualmente (APK externo), baja hasta *Ajustes avanzados* y activa "**Permitir ajustes restringidos**" para desbloquear la gestión completa de permisos en segundo plano.



¿Por qué Xiaomi es diferente?

MIUI y HyperOS (sucesor de MIUI) implementan una capa de gestión de energía llamada *Battery Saver Pro* que va más allá del estándar AOSP de Android. Esta capa mata procesos en segundo plano de forma agresiva para extender la batería, pero rompe funcionalidades de apps que dependen de servicios continuos (salud, geofencing, mensajería). **Sin los tres ajustes anteriores, el servicio de proximidad de Eco-Guía dejará de notificar tras 2-4 horas de inactividad**, incluso con la app abierta en segundo plano. Este problema NO ocurre en dispositivos Samsung, Pixel, Motorola, OnePlus o cualquier otro fabricante con Android stock.

— SECCIÓN 08

Solución de Problemas Comunes de Instalación

Diagnóstico rápido de errores típicos de instalación, compilación y ejecución en producción.

Esta sección documenta los cuatro errores más frecuentes reportados durante el despliegue de Eco-Guía, junto con su causa raíz y la solución paso a paso. Cada entrada sigue el formato **Problema → Causa → Solución** para permitir diagnóstico rápido sin necesidad de leer documentación extensa. Si tu error no aparece aquí, consulta el logcat en tiempo real con `adb logcat | grep EcoGuia` y revisa la sección de issues del repositorio GitHub.



Error: "App no instalada" o "Error al analizar el paquete"



Causa: Existe una versión previa instalada con una firma criptográfica diferente (por ejemplo, compilada desde otra computadora con un keystore distinto, o una versión Debug instalada previamente que ahora intentas sobrescribir con un APK Release).

Solución: Desinstala por completo la aplicación EcoGuia de tu teléfono (Ajustes → Aplicaciones → EcoGuia → Desinstalar) y vuelve a intentar la instalación del nuevo APK. Si persiste, ejecuta `adb uninstall com.ecoguia.app` desde la terminal para forzar la desinstalación completa incluyendo datos residuales.



Error: Gradle no genera el APK (`google-services.json missing`)



Causa: Falta el archivo de configuración de Firebase en el proyecto, o está colocado en una ruta incorrecta. Sin este archivo, el plugin `com.google.gms.google-services` no puede inyectar las claves API en los recursos de la app y la compilación falla en la tarea `processDebugGoogleServices`.

Solución: Descarga el archivo `google-services.json` desde la consola de Firebase (*Project Settings* → *Your apps* → *google-services.json*) y asegúrate de colocarlo dentro de las carpetas `mobile/` y `tv/` del proyecto. Verifica con `ls Eco-Guia/mobile/google-services.json` que el archivo existe físicamente antes de volver a compilar.

! Error: La terminal no reconoce el comando `adb`

✓ **Causa:** La ruta del Android SDK Platform Tools no está agregada a la variable de entorno PATH del sistema operativo. Esto es común en instalaciones frescas de Android Studio donde el SDK se descarga pero no se configura globalmente.

Solución: Utiliza la ruta completa al binario:

- **Windows:** `C:\Users\<TuUsuario>\AppData\Local\Android\Sdk\platform-tools\adb.exe install ...`
- **Linux/macOS:** `~/Android/Sdk/platform-tools/adb install ...`

O simplemente transfiere el archivo .apk a la memoria de tu teléfono mediante el explorador de archivos e instálalo directamente (ver §5.3 Alternativa 2). Para configurar PATH permanentemente, agrega `export PATH=$PATH:~/Android/Sdk/platform-tools` a tu `.bashrc` o `.zshrc`.

! El servicio de proximidad no notifica con la pantalla apagada

✓ **Causa:** El sistema operativo cerró el proceso en segundo plano por ahorro de energía. Esto es especialmente común en dispositivos Xiaomi/Redmi/POCO con MIUI o HyperOS, donde la capa *Battery Saver Pro* mata servicios en segundo plano agresivamente tras 2-4 horas de inactividad de pantalla.

Solución: Revisa la sección **7.3** de este manual y aplica los tres ajustes indispensables para dispositivos Xiaomi: (1) Autostart, (2) Sin Restricciones de Batería, (3) Permitir Ajustes Restringidos. Para otros fabricantes (Samsung, Pixel), verifica que la optimización de batería esté desactivada para Eco-Guía en **Ajustes → Aplicaciones → EcoGuía → Batería**.

i Diagnóstico avanzado con logcat

Si ninguna de las soluciones anteriores resuelve tu problema, conecta el teléfono por USB y ejecuta `adb logcat | grep EcoGuía` mientras reproduces el error. Los logs en tiempo real mostrarán excepciones Java/Kotlin, errores de permisos y fallos de Firebase que no son visibles en la UI de la app. Adjunta el log completo (con timestamps) al abrir un issue en GitHub para acelerar el diagnóstico.

— SECCIÓN 09

Glosario Técnico

Definiciones de los términos técnicos, acrónimos y conceptos utilizados a lo largo del manual.

Este glosario recopila los términos técnicos y acrónimos utilizados en este manual, ordenados alfabéticamente. Las definiciones están orientadas al contexto específico de Eco-Guía y del ecosistema Android; pueden tener significados más amplios en otros dominios. Para términos no incluidos aquí, consulta la documentación oficial de Android Developers o Firebase.

Android Debug Bridge

ADB

Herramienta de línea de comandos incluida en Android SDK Platform Tools que permite a una computadora comunicarse con un dispositivo Android para instalar apps, leer logs, transferir archivos y ejecutar comandos del sistema. Es el canal estándar para depuración USB.

Android Package

APK

Formato de archivo comprimido (basado en ZIP) que contiene el código compilado, recursos, manifiesto y firma criptográfica de una aplicación Android. Es el artefacto instalable directamente en cualquier dispositivo Android.

Android Studio

IDE

Entorno de desarrollo integrado oficial de Google para aplicaciones Android, basado en IntelliJ IDEA. Incluye editor de código, emulador, depurador, sistema de build Gradle y herramientas de profiling.

Android TV

ATV

Variante del sistema operativo Android optimizada para televisores y set-top boxes. Eco-Guía incluye un módulo `tv` con interfaz adaptada a control remoto y visualización panorámica.

ARCore

AR

SDK de Realidad Aumentada de Google que permite a las apps Android detectar superficies planas, estimar iluminación y rastrear movimiento del dispositivo para superponer contenido digital sobre el feed de cámara.

Firebase

FBS

Plataforma backend de Google que proporciona autenticación, base de datos en tiempo real, almacenamiento, analítica y mensajería push. Eco-Guía utiliza Firebase como capa de datos compartida entre los tres módulos.

Geofencing

GEO

Técnica de localización que define perímetros virtuales alrededor de puntos geográficos. Cuando el dispositivo entra o sale del perímetro, el sistema operativo notifica a la app, incluso si está en segundo plano. Eco-Guía usa geofences de 50m para alertas de proximidad histórica.

Google Play Protect

GPP

Servicio antimalware integrado en todos los dispositivos Android certificados por Google que analiza apps instaladas (de Play Store o de fuentes externas) usando heurísticas en la nube. Puede bloquear APKs Debug no firmados con keystore propio.

Gradle

GRADLE

Sistema de automatización de builds utilizado por Android Studio para compilar código, resolver dependencias, procesar recursos y empaquetar APKs. El comando `gradlew` es el wrapper que descarga la versión correcta de Gradle automáticamente.

Java KeyStore

JKS

Formato de archivo de almacén de claves criptográficas de Java. Contiene pares de claves públicas/privadas utilizadas para firmar APKs de release. La pérdida del archivo `.jks` impide publicar actualizaciones de la app con el mismo package name.

MIUI / HyperOS

MIUI

Capa de personalización propietaria de Xiaomi instalada sobre Android en dispositivos Xiaomi, Redmi y POCO. HyperOS es la versión renombrada introducida en 2024. Ambas implementan gestión agresiva de batería que requiere ajustes especiales para apps con servicios en segundo plano.

ProGuard / R8

R8

Herramientas de ofuscación y optimización de bytecode integradas en el build de Android que reducen el tamaño del APK, eliminan código muerto y renombran clases/métodos para dificultar el reverse engineering. R8 es el sucesor moderno de ProGuard.

USB Debugging

USBDBG

Modo oculto de Android que permite a una computadora conectada por cable ejecutar comandos ADB sobre el dispositivo. Se activa pulsando 7 veces "Número de compilación" en Ajustes → Acerca del teléfono, y luego habilitando la casilla correspondiente en Opciones de desarrollador.

Wear OS

WEAR

Versión de Android optimizada para smartwatches y wearables. Eco-Guía incluye un módulo `wear` con notificaciones de proximidad y accesos rápidos accesibles desde la muñeca, sin necesidad de sacar el teléfono del bolsillo.

— SECCIÓN 10 · CIERRE

Resumen y Próximos Pasos

Recapitulación de los tres métodos y referencias para profundizar.

Este manual ha cubierto de forma exhaustiva los tres métodos disponibles para instalar y desplegar la aplicación Eco-Guía en sus tres módulos (móvil, TV y Wear OS), desde la preparación del entorno de desarrollo hasta la configuración post-instalación en dispositivos con capas de personalización agresivas. La elección del método debe basarse en el público objetivo: **Método 1 (Android Studio)** para desarrollo iterativo del equipo interno, **Método 2 (APK Debug)** para distribución a evaluadores y QA sin acceso al código fuente, y **Método 3 (APK Release firmado)** para producción y publicación formal en Google Play Store.

Como próximos pasos, se recomienda: (1) **automatizar los builds Release** mediante un pipeline CI/CD en GitHub Actions para garantizar reproducibilidad y trazabilidad, (2) **documentar el keystore** en un gestor seguro y compartir acceso solo con personal autorizado, (3) **monitorear crashlytics** en Firebase para detectar tempranamente errores en producción, y (4) **actualizar este manual** con cada release mayor del proyecto, agregando nuevas secciones de troubleshooting según se reporten issues en campo.

Fin del Manual Técnico Eco-Guía v1.0.0

AGOSTO 2026 · ECOSISTEMA ANDROID